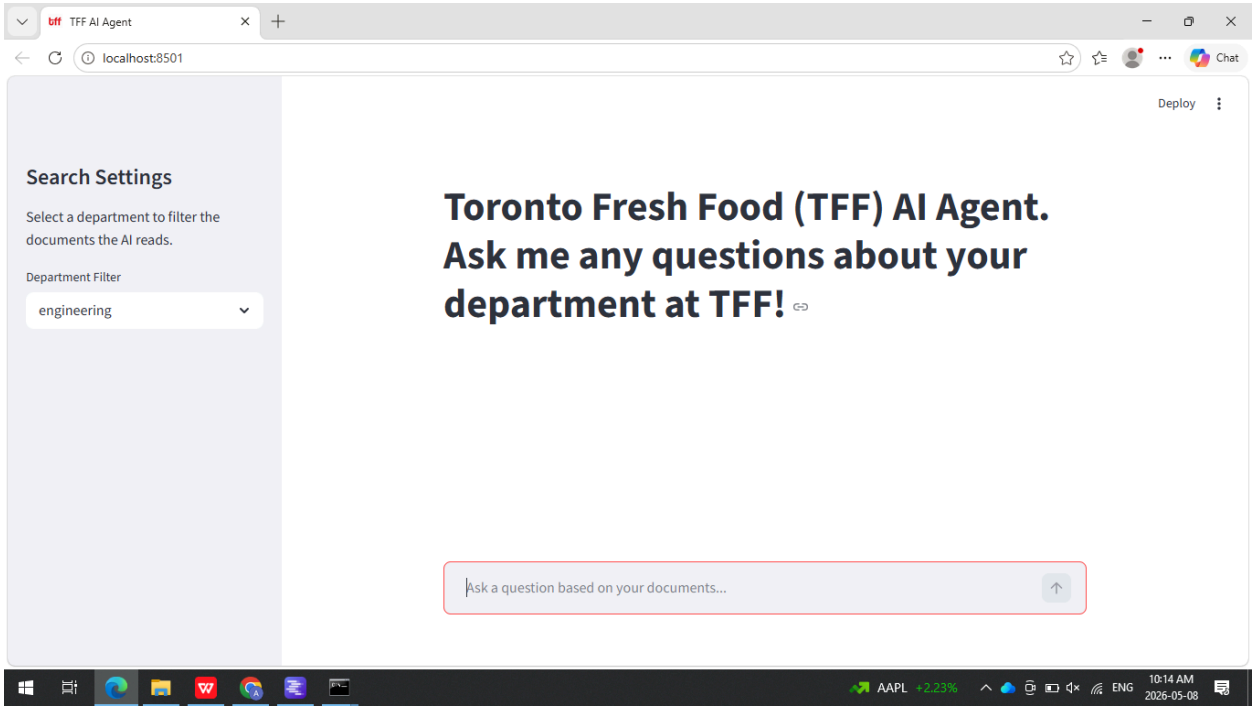
Toronto Fresh Food (TFF)

Local LLM AI Agent Tutorial

June 1, 2026

Prototype of the User Interface

The following image showcases a prototype of what the blank user interface (UI) would look like upon running Streamlit. You can see that the “TFF Logo.jpg” image was used as the icon for the UI to help brand the AI agents.



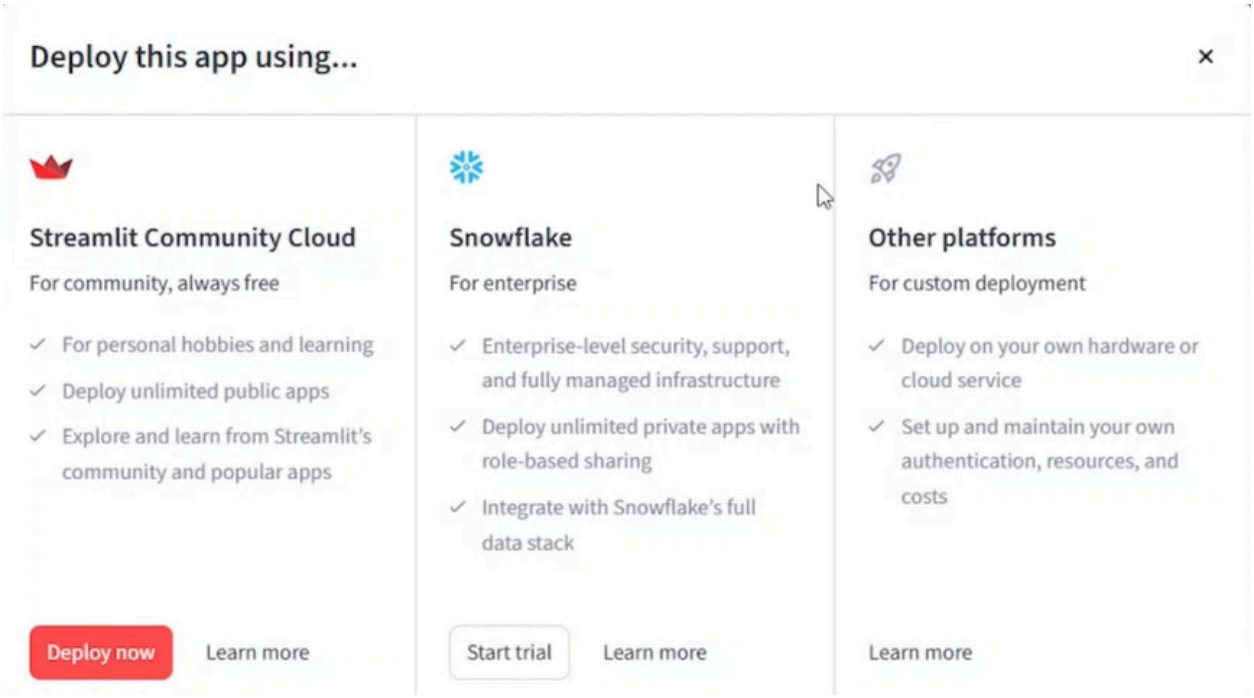
“Demo Video #1” from the GitHub files showcases all of the features that the Streamlit user interface has to offer, you can look at the following section to get a textual description of each feature.

Department Filter: The panel underneath the department filter allows you to choose what department’s files you want the AI to use when responding to your answer. It ensures that all 8 of the department retrieval-augmented generation (RAG) pipelines remain separate.

The three dots: The three dots allow you to customize the appearance of the user interface.

- You can choose whether or not you want a light or dark theme or keep the same theme as the rest of your device
- Rerun immediately halts the current script execution and restarts it from the top. This allows for the app to be updated with new data, state changes or UI elements

- You can turn on or off auto rerun
- Clear cache removes stored data or resources from memory, forcing the app to re-run decorated functions (`@st.cache_data` or `@st.cache_resource`) and fetch fresh data. It is used to update outdated information, free-up memory or debug, and can be done via the UI menu or programmatically.
- The print function outputs text to the server-side terminal (console) where the application is running, not to the web browser screen. It is useful for debugging in the backend but it is not visible to users in the rendered web app
- You can record your screen with or without audio by hitting the record screen button
- The deploy button opens up a menu (see in the video) of options you can use to share your AI agent:



- I don't recommend using Streamlit's deploy options because it is more meant for sharing prototypes than actually sharing a local LLM model. Refer to the "Physical Solution vs Cloud Solution" section in the README document to figure out how to deploy the local LLM using Runpod

Demo That Showcases The RAG Pipelines Working Correctly

In order to test that the AI model is not hallucinating (see the README document to learn more about hallucinating from an AI standpoint) we need to make two documents with metadata from two separate departments. We will then load them into ChromaDB using the EmbeddedModel.py code we made earlier. These two text documents must contain two elements so that the RAG pipelines are working correctly:

1. At least one of the documents must contain a false fact. We want the AI to suppress itself and try and spit out a false answer from the document rather than spitting out a "hallucinated" answer based on its prior training
2. The two documents must contradict each other, ensuring that the AI will only take the statement from the document with the appropriate metadata. This ensures that our metadata filtering system is working correctly

I created two .txt documents to test out of AI Agents. The documents are listed below:

1. Document to Test RAG and Metadata Filtering (Engineering).txt

2. Document to Test RAG and Metadata Filtering (Accounting).txt

As you can see, these two documents fulfill both the criteria above:

1. The second document says that accounts receivable are amounts of money that we owe someone else, this is false and should have been written as accounts payable instead

2. The first document says that we owe $750,000 to the manufacturing company while the second document says that we owe $1,000,000 to the manufacturing company

After following these steps in the README document turn these files them vectors and store them in ChromaDB using the EmbeddedModel.py code, I now needed to come up with two prompts that the AI would use to generate the desired response:

The prompt for the engineering department: “How much do we owe the manufacturing company for making the automated production line?”

The prompt for the accounting department: “What are the accounts receivable for the next month? How much do we owe the manufacturing company?”

- As you can see from “Demo Video #2”, the AI agent stated that we owed the manufacturing company $750,000 dollars when searching through the engineering department’s documents
- However in “Demo Video #3”, the AI agent stated that we owed the manufacturing company $1,000,000 dollars, as stated in the second document. It also stated that the $50,000 owed to the sushi supplier and the $1,000,000 dollars owned to the manufacturing company were accounts receivable rather than accounts payable when searching through the accounting department’s documents
- As a result, these tests prove that our local LLM can store documents in their own separate RAG pipelines because when asked to look through the engineering departments documents, it only looked through the engineering departments documents and when asked to look through the accounting departments documents, it only looked through the accounting departments documents
- It also proved that AI agent will suppress its own knowledge and won't hallucinate when generating prompts because it stated that amounts owed are a part of accounts receivable instead of stating that they are a part of accounts payable, which is what the AI was trained to say